

GUÍA PRÁCTICA | ERP CONTABLE

APIs REST con Node, Express y Vue 3

Programación asíncrona ES6+, consumo de servicios con Fetch/Axios, componentes y reactividad.

Vue + Vuetify
localhost:5173



Express + JSON
localhost:3000

Contactos y movimientos contables

Material didáctico para transformar el proyecto erp-contable. Los datos iniciales son simulados y viven en memoria.

1. Resultado y arquitectura

El proyecto tendrá dos aplicaciones independientes. Vue muestra la interfaz; Express recibe solicitudes, valida datos y responde JSON.

```
VUEBASICO/  
├─ erp-contable/      # Frontend Vue + Vite  
└─ erp-api/           # Backend Node + Express
```

Regla de terminal: ejecuta cada comando dentro de la carpeta que contiene el `package.json` correspondiente.

2. Crear el backend Express

1. Abre CMD y entra en la carpeta principal.
2. Crea el backend y prepara npm.
3. Instala Express, CORS y Nodemon.

```
cd /d "C:\Users\jdcad\Materias\Talleres\interfaces web 2026-2\PROYECTO\VUEBASICO"  
mkdir erp-api  
cd erp-api  
npm init -y  
npm install express cors  
npm install --save-dev nodemon
```

En el `package.json` del backend agrega:

```
"scripts": {  
  "start": "node server.js",  
  "dev": "nodemon server.js"  
}
```

Crea `server.js`:

```

import express from 'express'
import cors from 'cors'

const app = express()
const PORT = 3000

app.use(cors())
app.use(express.json())

let contactos = [
  { id: 1, nombre: 'Ana Torres', correo: 'ana@erp.test', tipo: 'Cliente' },
  { id: 2, nombre: 'Luis Perez', correo: 'luis@erp.test', tipo: 'Proveedor' }
]

let movimientos = [
  { id: 1, concepto: 'Venta de servicios', tipo: 'Ingreso', monto: 1500 },
  { id: 2, concepto: 'Compra de insumos', tipo: 'Egreso', monto: 450 }
]

app.get('/api/contactos', (req, res) => {
  res.status(200).json(contactos)
})

app.get('/api/contactos/:id', (req, res) => {
  const contacto = contactos.find(item => item.id === Number(req.params.id))
  if (!contacto) return res.status(404).json({ mensaje: 'Contacto no encontrado' })
  res.status(200).json(contacto)
})

app.post('/api/contactos', (req, res) => {
  const { nombre, correo, tipo } = req.body
  if (!nombre || !correo || !tipo) {
    return res.status(400).json({ mensaje: 'Faltan datos obligatorios' })
  }
  const nuevoContacto = { id: Date.now(), nombre, correo, tipo }
  contactos.push(nuevoContacto)
  res.status(201).json(nuevoContacto)
})

app.delete('/api/contactos/:id', (req, res) => {
  const indice = contactos.findIndex(item => item.id === Number(req.params.id))
  if (indice === -1) return res.status(404).json({ mensaje: 'Contacto no encontrado' })
  contactos.splice(indice, 1)
  res.status(204).send()
})

app.get('/api/movimientos', (req, res) => {
  res.status(200).json(movimientos)
})

app.listen(PORT, () => {
  console.log(`API disponible en http://localhost:${PORT}`)
})

```

Qué hacen los middlewares: `cors()` permite la comunicación entre puertos distintos y `express.json()` convierte el body JSON en `req.body`.

3. Ejecutar y probar Express

```
cd /d "C:\Users\jdcad\Materias\Talleres\interfaces web 2026-2\PROYECTO\VUEBASICO\erp-api"
npm run dev
```

Debe aparecer **API disponible** en **http://localhost:3000**. Mantén esa terminal abierta.

Método	Endpoint	Uso	Éxito
GET	/api/contactos	Listar contactos	200
GET	/api/contactos/1	Consultar uno	200
POST	/api/contactos	Crear contacto	201
PUT	/api/contactos/1	Actualizar contacto	200
DELETE	/api/contactos/1	Eliminar contacto	204
GET	/api/movimientos	Listar movimientos	200

Pruebas en Postman o Insomnia

Para crear un contacto usa **POST** **http://localhost:3000/api/contactos**, header **Content-Type: application/json** y este body:

```
{
  "nombre": "Maria Lopez",
  "correo": "maria@erp.test",
  "tipo": "Cliente"
}
```

Prueba también un ID inexistente para observar **404 Not Found** y un body incompleto para observar **400 Bad Request**.

4. Programación asincrónica ES6+

Una solicitud HTTP tarda un tiempo impredecible. JavaScript usa promesas para continuar sin congelar la interfaz.

Promesas

```
fetch('http://localhost:3000/api/contactos')
  .then(respuesta => respuesta.json())
  .then(datos => console.log(datos))
  .catch(error => console.error(error))
```

- **pending:** esperando.
- **fulfilled:** terminó correctamente.
- **rejected:** terminó con error.

async y await

```
async function cargarContactos() {
  const respuesta = await fetch('http://localhost:3000/api/contactos')
  const datos = await respuesta.json()
  return datos
}
```

`await` pausa esa función asincrónica, pero no bloquea todo el navegador.

try/catch/finally

```
async function cargarContactos() {
  cargando.value = true
  error.value = ''
  try {
    const respuesta = await fetch('http://localhost:3000/api/contactos')
    if (!respuesta.ok) throw new Error(`HTTP ${respuesta.status}`)
    contactos.value = await respuesta.json()
  } catch (problema) {
    error.value = problema.message
  } finally {
    cargando.value = false
  }
}
```

Importante: Fetch no rechaza automáticamente la promesa con un 404 o 500. Comprueba siempre `respuesta.ok`.

5. Consumir la API desde Vue

En `Clientes.vue`, el estado y la carga pueden organizarse con Composition API:

```

<script setup>
import { onMounted, ref } from 'vue'

const contactos = ref([])
const cargando = ref(false)
const error = ref('')

async function cargarContactos() {
  cargando.value = true
  error.value = ''
  try {
    const respuesta = await fetch('http://localhost:3000/api/contactos')
    if (!respuesta.ok) throw new Error('No se pudieron cargar los contactos')
    contactos.value = await respuesta.json()
  } catch (problema) {
    error.value = problema.message
  } finally {
    cargando.value = false
  }
}

onMounted(cargarContactos)
</script>

<template>
<p v-if="cargando">Cargando...</p>
<p v-else-if="error">{{ error }}</p>
<ul v-else>
  <li v-for="contacto in contactos" :key="contacto.id">
    {{ contacto.nombre }} - {{ contacto.correo }}
  </li>
</ul>
</template>

```

La asignación `contactos.value = datos` cambia el estado reactivo y Vue actualiza el `v-for` automáticamente.

6. Crear datos y comparar Fetch/Axios

```

const nuevoContacto = ref({ nombre: '', correo: '', tipo: 'Cliente' })

async function crearContacto() {
  const respuesta = await fetch('http://localhost:3000/api/contactos', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(nuevoContacto.value)
  })
  if (!respuesta.ok) throw new Error('No se pudo crear el contacto')
  const creado = await respuesta.json()
  contactos.value.push(creado)
}

```

Fetch	Axios
Incluido en el navegador.	Se instala con <code>npm install axios</code> .
Hay que revisar <code>respuesta.ok</code> .	Interpreta JSON y errores HTTP con comodidad.
Buena opción para empezar.	Útil con muchas rutas, interceptores y configuración común.

```
import axios from 'axios'
const api = axios.create({ baseUrl: 'http://localhost:3000/api' })
const respuesta = await api.get('/contactos')
contactos.value = respuesta.data
```

7. Componentes y reactividad

Divide la interfaz por responsabilidades: `ContactoForm.vue`, `ContactosTabla.vue`, `MovimientoForm.vue` y `TarjetaKPI.vue`.

- **ref**: listas, textos y booleanos reactivos.
- **computed**: valores derivados y calculados.
- **onMounted**: carga inicial.
- **Props**: datos del padre al hijo.
- **Eventos**: acciones del hijo al padre.

```
const totalIngresos = computed(() => {
  return movimientos.value
    .filter(movimiento => movimiento.tipo === 'Ingreso')
    .reduce((total, movimiento) => total + movimiento.monto, 0)
})
```

Cuando cambia `movimientos.value`, Vue recalcula el total y actualiza las tarjetas del dashboard.

8. Orden de trabajo y comprobación

Abre dos terminales:

```
# Terminal 1: backend
cd /d "C:\Users\jdcad\Materias\Talleres\interfaces web 2026-2\PROYECTO\VUEBASICO\erp-api"
npm run dev

# Terminal 2: frontend
cd /d "C:\Users\jdcad\Materias\Talleres\interfaces web 2026-2\PROYECTO\VUEBASICO\erp-contable"
npm run dev
```

- La API responde en `http://localhost:3000`.
- Vue responde en `http://localhost:5173`.
- La lista de contactos llega desde Express.
- Un error de red se muestra en la interfaz.

- Crear un contacto actualiza la tabla reactivamente.
- Los movimientos alimentan los totales con `computed`.

9. Errores frecuentes

Error	Solución
<code>ENOENT package.json</code>	Entra en <code>erp-api</code> o <code>erp-contable</code> .
CORS	Confirma <code>app.use(cors())</code> y que Express esté encendido.
Failed to fetch	Revisa puerto, URL y servidor.
Vue no actualiza	Usa <code>contactos.value</code> si es un <code>ref</code> .
Datos desaparecen	Es normal: están en memoria; luego agrega base de datos.

Actividad: crea dos contactos, carga la lista desde Vue, registra un ingreso y un egreso, calcula el saldo con `computed` y simula apagar Express para mostrar un mensaje de error.